

GROUP 4 FINAL REPORT

BAKER JESSICA, HU ZIHAN, MIYAJI TSUKASA, VILLANUEVA LUKE

Abstract

Based on the data from Fingerhut, this project primarily aims to achieve three main objectives: improve purchase rate, improve activation rate, and find the most optimal journeys. Before further analysis, our groups preprocessed the original data set with the following two steps: first, we cleaned the whole data to remove possible duplicates. Second, we removed unreasonable records due to network delay, which can be easily done by the timestamp column (as stated by Ben in the Week 2 meeting). After that, our group built various models based on the three goals in mind. To begin with, our group built an Apriori model to get a high level idea of the whole dataset, where our group found that since the data is in time series format, time series analysis may be applied for further evaluation purposes. Then we built an RNN (Recurrent Neural Network) model for binary classification, which achieved an accuracy of approximately 90%. Though the RNN performed well on the sample set, because we cannot extract feature importance from this model, our group decided to set it as a model for evaluation, which could further prove the results of our other models. Then our group developed models that analyzed the composition of the journey events of two different customer demographics. Inspecting customers on their path to purchase and their path to activation. These models include logistic regression, Markov Chain analysis, and XGBoost. Based on the performance of these models, our group finds that there are certain conclusions to be made from the original data set, as shown in the conclusion section.

Introduction

In the evolving e-commerce market, Fingerhut has established itself as a niche online retailer which offers a unique purchasing model that allows customers with weak or no credit history to acquire goods through its FreshStart program. The FreshStart program offers customers a line of credit to be used to purchase goods on the FingerHut website. These individuals can then pay off the loan monthly and can receive a higher credit limit in the future by making full payments on time. It is important for Fingerhut to look at the website journeys of their FreshStart customers to identify ways in which they can improve the customer experience and lead more people to purchase – ultimately with the goal of building their customers' credit.

Our analysis focuses on the customer journeys derived from the clickstream data from Fingerhut's FreshStart customers recorded from 2021 to 2023. Our objective is to extract the most influential events that impact the customer's ability to reach their goals. Meaning, we are finding the events that increase/decrease the likelihood to activate their to purchase a Fingerhut product and to activate their account. Furthermore, we also inspect for 'optimal' journeys. We define optimal as the least amount of steps that lets an activated customer to a purchase. To gain insight from this data and make predictions on a customer's journey we implemented a variety of models. The models used in our analysis are logistic regression, random forest classification, markov chain analysis, XGBoost, and finally a recurrent neural network for evaluation. This paper will dedicate sections to each portion of our comprehensive analysis, from our methodology, to our analysis of different aspects of the journey, and finally strategic recommendations.

Methodology

Methods being implemented in this project consists of the following:

- The Apriori algorithm:
 - It is a foundational approach used in data mining to extract frequent itemsets from large databases and, subsequently, to derive association rules.
- Logistics / Linear Regression:
 - The most frequently used algorithms in this project because of its simplicity and explainability due to its linearity. This is used for classification / regression.
- XGBoost:
 - The core principle of XGBoost lies in sequentially building a series of weak learners—typically decision trees—where each new tree attempts to correct the errors made by the previous ones.
- Random forest:
 - It builds on the concept of decision trees, but rather than relying on a single decision tree, it combines the predictions from multiple decision trees to make a more accurate and robust prediction, much like XGBoost.
- Markov chain analysis:
 - It is based on the Markov property, which stipulates that the probability of transitioning to any future state depends only on the current state and not on the

sequence of events that preceded it, since we are having a time series data set, this methodology might be of help.

- Recurrent Neural Network (RNN):
 - RNN is an extension of the traditional neural network with a recurrent loop inside the hidden layer, designed to effectively capture temporal dynamic behaviors. This makes it particularly suitable for analyzing time series data, such as the data set we are facing.

Data Overview and Initial Analysis

Overview:

The Fingerhut dataset serves as a foundational resource for analyzing customer interactions. It records a variety of identifiers, customer IDs, account IDs, event IDs, along with timestamps and journey step sizes, offering a granular view of the frequency and timing with which customers engage in specific events. Despite its potential, the dataset is marred by the presence of duplicate entries, a flaw attributed to the web tracking system's susceptibility to errors, such as double-firing events. These redundancies not only inflate the data but also obscure the true nature of customer journeys, presenting a significant hurdle for any meaningful analysis. In response to these challenges, a data cleaning and transformation process is important to enhance the dataset's analytical value.

The first step in this process is the removal of duplicate records, with particular attention paid to those instances where all data points are identical except for the sequence of journey

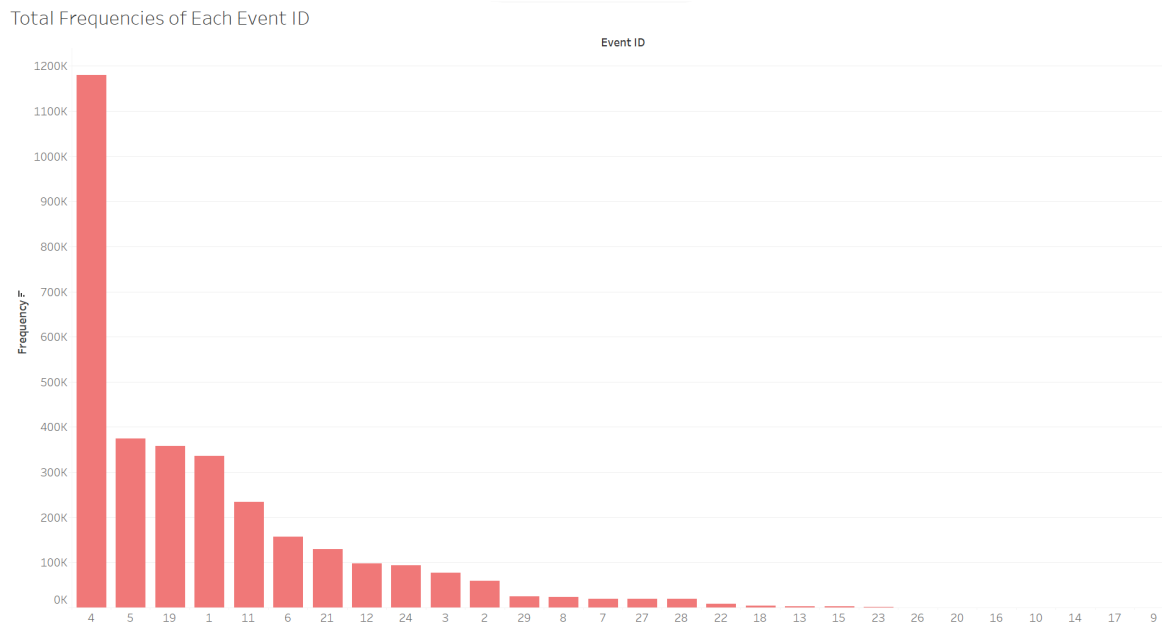
steps. This targeted approach ensures that only unique customer interactions are retained, thereby eliminating artificially inflated engagement metrics caused by the tracking errors. Following the cleaning of the dataset, further modifying is achieved by discarding columns that do not contribute to the core analysis objectives. Specifically, the 'journey step size' and 'timestamp' columns are removed. The removal of these details focuses the dataset more squarely on the interaction events themselves, rather than the sequential nuances of those interactions. To add analytical depth, a new 'count' column is introduced. This column quantitatively captures the frequency of each specific event for individual customers. This addition shifts the dataset from a mere log of interactions to a record that highlights the customer engagement for each event more. This table reorganizes the data to display the event count for each event by customer, offering a comprehensive snapshot of how different segments of the customer base interact with various elements of the business.

Furthermore, the following table describes the overall composition of events within the data.

From the table, it can be concluded (from the given event definitions of each event) that *browse_products*, *view_cart*, *application_web_view*, *promotion_created*, and *add_to_cart* are the top five most popular events, respectively.

Initial analysis: Apriori

As previously mentioned, our goal was to find what influenced the customer journey. Our initial hypothesis was that the types of events had a connection to a specific set of events. To get a general sense of how the data behaved, the apriori algorithm was implemented. This algorithm let us see which events tended to exist together in the customer's journey and which events would lead to another. This analysis aided in deciding which model to use next.



The following snippet shows a preview of the apriori algorithm.

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage
5880916	(2, 3, 7, 11, 27, 29)	(1, 4, 5, 8, 12, 19, 28)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5946938	(2, 3, 7, 11, 27)	(1, 4, 5, 6, 8, 12, 19, 28, 29)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5941746	(2, 3, 6, 7, 11, 27, 29)	(1, 4, 5, 8, 12, 19, 28)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5506191	(2, 3, 7, 11, 27)	(1, 4, 5, 8, 12, 19, 28)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5816744	(2, 3, 7, 11, 27)	(1, 4, 5, 6, 8, 12, 19, 28)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5882244	(2, 3, 7, 11, 27)	(1, 4, 5, 8, 12, 19, 28, 29)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5944602	(2, 3, 6, 7, 11, 27)	(1, 4, 5, 8, 12, 19, 28, 29)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694
5815380	(2, 3, 6, 7, 11, 27)	(1, 4, 5, 8, 12, 19, 28)	0.089883	0.122452	0.0627	0.697575	5.69674	0.051694

From this, we see that some events most definitely lead to another event. So, this proves our initial hypothesis. To prove this further mathematically, classification and regression models would be developed to predict customer demographic (based on purchase and activation) and journey length (journey steps). These models would show feature importance in the customer journey.

Further Data Analysis

Logistic regression/linear regression

→ Initial Purchase Analysis

One important factor that we decided to delve into was how to increase customer purchases. To do this, we decided to use a logistic regression model. For the data, we had to deal with which demographic of customer we wanted to analyze. For the sake of simplicity, we decided to include all types of demographics.

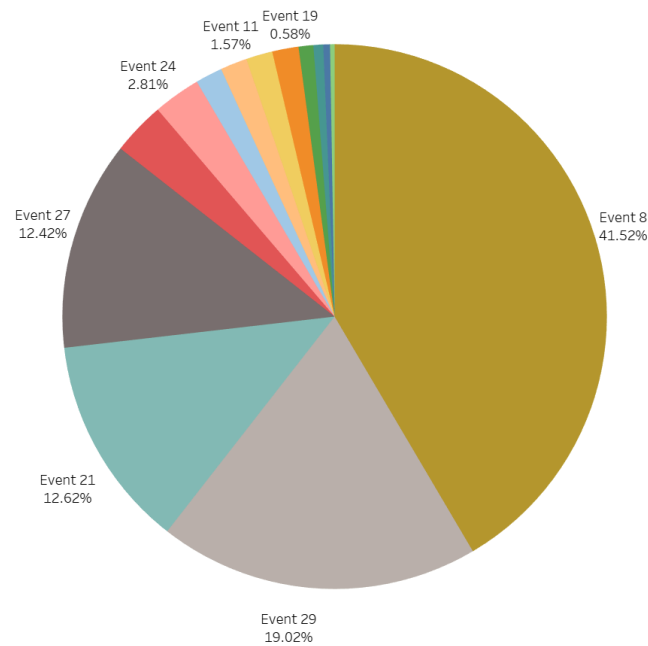
First, we would fit the model to the data to predict whether a customer would purchase a product, and we would tune it to increase its accuracy. After obtaining an accurate predictive model, we would extract the coefficients to interpret each event's influence on a customer's likelihood to purchase a Fingerhut product.

To start, we made a binary response variable based on whether event 18 or 7 existed in the customer's journey. Tests of collinearity proved events 9 and 17 are zero variance and should be omitted. Also, events 10 and 20 do not exist for any customers. In addition, events like 12, 15, and 27 are highly collinear, so a general logistic regression model cannot be used. Hence, the *glmnet* engine was used instead to bypass the multicollinearity issue. After normalizing the data, fitting the model and predicting resulted in the following metrics:

- F Score - 0.9785664
- Accuracy (classification matches divided by total customers) - 0.9662663

The metrics imply that the model is 96% accurate. This model's coefficients are then extracted and interpreted. The following chart showcases the weight of the most important variables to the probability of a customer purchasing a product:

Event Types' Influence to Classification Algorithm



Essentially, the pie chart shows the most important variables that lead to a customer purchasing. More analysis of this will be discussed in this section's conclusion. The following snippet is the full detailed breakdown of the coefficients to each variable in the logistic regression model:

	term <chr>	estimate <dbl>	penalty <dbl>
1	X8	2.92	0
2	(Intercept)	-1.77	0
3	X29	1.18	0
4	X27	0.882	0
5	X21	-0.642	0
6	X24	0.209	0
7	X1	-0.206	0
8	X22	-0.124	0
9	X4	0.0859	0
10	X19	0.0858	0
11	X3	-0.0693	0
12	X6	-0.0215	0
13	X2	-0.0112	0
14	X5	-0.00207	0
15	X11	0	0

We will breakdown the interpretation to the first five entries:

- X8
 - This is event 8. Since this event has the highest coefficient and is positive, this implies that event 8 contributes the most likelihood to a customer purchasing a product. In other words, placing a downpayment highly increases the chances of a customer purchasing.
- Intercept
 - The intercept tells the default chance a customer has of purchasing. Since the value is negative, this implies the base chance of a customer purchasing is less than 50/50.

- In detail, the intercept tells the default log odds of customer purchasing.

So, $e^{-1.77} = 0.17033$. This is the odds of a customer purchasing immediately after activation.

- X29

- This is event 29. This is the next most influential event to customer purchasing. This implies that account activation is the second most important factor to purchasing.

- X27

- This is event 27. This implies that clearing a downpayment is the third most important factor to purchasing.

- X21

- This is event 21. Since this is negative, *catalog_mail* actually decreases the chances a customer would purchase. Since its magnitude is fourth, it isn't as impactful as the previous factors, but it should still be considered nonetheless.

In conclusion, Fingerhut should encourage customers to complete events *place_downpayment*, *account_activation*, *account_downpaymentcleared*, and *camapign_emailclicked*. The events that Fingerhut should discourage its customers from completing because they have a negative correlation with making a purchase are *catalog_mail*, *promotion_created*, and *pre_application_3rd_party_affiliate*. It is interesting that the promotion being created, and a catalog sent by mail discourages customers from purchasing. Fingerhut should pay attention to this result because it seems that expanding these resources of creating promotion and sending a catalog is not having the intended effect of encouraging a customer to make a purchase.

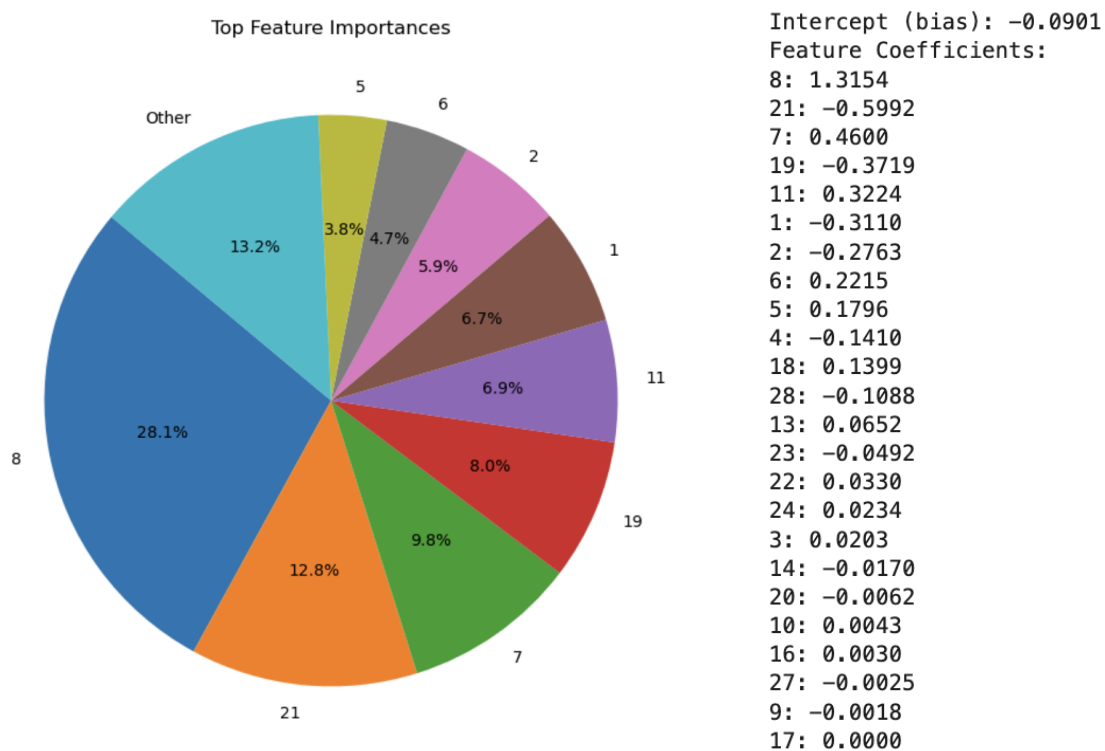
→ Initial Activation Analysis

The next question we posed was how can Fingerhut encourage customers to activate their accounts. An “activated” account typically means that the account holder has met the initial requirements set by Fingerhut’s FreshStart program to activate their account. This typically means making a one-time purchase up to a certain credit limit and paying off this purchase in installments. Activation is one step on the way to graduating to a standard Fingerhut credit account. Thus we aim to identify events that positively and negatively correlate with account activation. To explore this question, we implemented a couple of techniques. First, similar to above, we trained a logistic regression model to predict a customer’s account activation based on the existence of particular events in their journey. After this, we implemented a Markov chain analysis to produce a heatmap of the probability of going from one event to the next for customers who did activate their account and customers who didn’t.

To preprocess the data for modeling, we included a ‘new_session’ variable indicating the start of a new session based on account_id change. Sessions were truncated after event_id = 29, and whole sessions were kept only in the absence of event_id = 29 and discarded sessions from previously activated accounts. The dataset was restructured to represent each session as a row, with columns for each event_id marked as 0 or 1 to denote the occurrence of events, and an additional column to indicate the activation status of the session. Columns 29, 12, and 15 were dropped as they are directly associated with account activation. The final cleaned dataset comprised 23.4% activated sessions and 76.6% non-activated sessions.

Training the logistic regression model proved to have some difficulties associated with it. The first was the imbalance of the activated and not activated sessions in the data. To account for this the model was trained on a balanced set of the data and tested on the set with the original

proportions. After hyperparameter tuning and balancing of the classification threshold – the best model achieved accuracy of 81.5% and an F-score of 60.5%. These values are not as high as the other models achieved for our data, and this could be due to a couple of reasons. Due to the linear nature of logistic regression models, it could be that the relationships in the data are too complex for this simple model to fully capture the relationship between feature existence and account activation. Additionally, while Lasso regression was utilized to perform some level of feature selection, other methods such as XGBoost perform a much more robust version of this selection. Regardless – the feature importances and coefficients can still provide a great jumping off point for further analysis.



We see that the top 5 most important features for predicting account activation are 8, 21, 7, 19, and 11. These are the event id's for place_downpayment, catalog_mail, place_order_web,

application_web_view, and add_to_cart, respectively. It is important to note the sign of these coefficients as well. Note that the intercept is negative, meaning the default chance of a customer activating their account is unlikely, as with purchase.

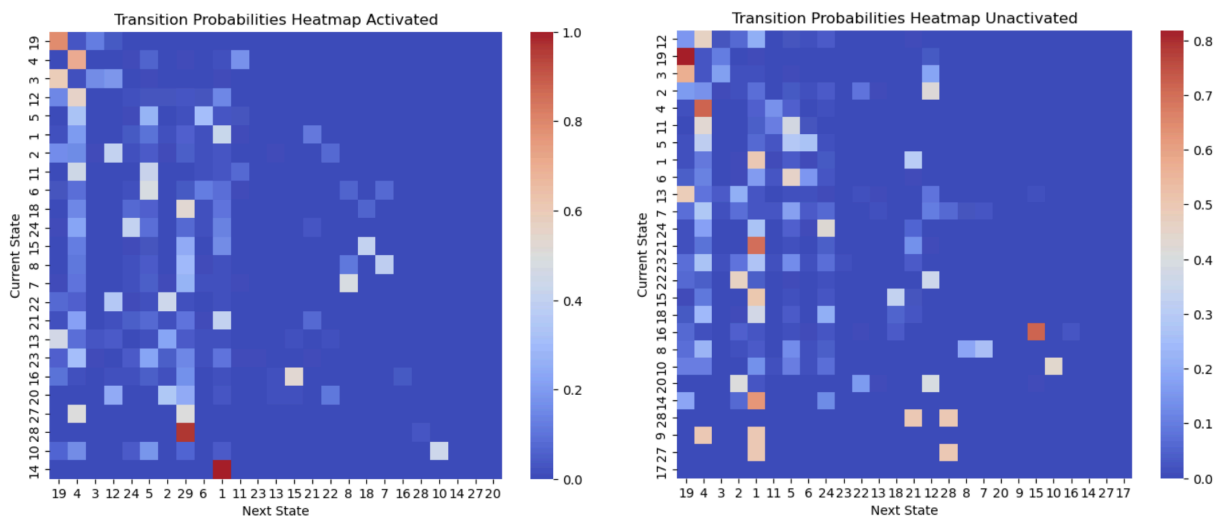
- Event 8
 - Similar to purchase, place_downpayment has the highest feature coefficient. This suggests that a customer is more likely to activate their account after having placed a down payment. Fingerhut should encourage customers to submit a down payment as soon as possible within their journeys.
- Event 21
 - Again, consistent with the results for purchase prediction, catalog_mail has a negative coefficient, meaning it decreases the customer's likelihood of account activation. As we will discuss later, this could be a place to start for Fingerhut to cut down on expending resources which don't actually lead to encouraging a customer's activation.
- Event 7
 - Place_order_web has a positive correlation with predicting a customer's account activation. This intuitively makes sense as well, because account activation should occur after the first purchase.
- Event 19
 - Application_web_view has a negative correlation with account activation. Something about viewing the app makes a customer less likely to eventually activate their account. Intuitively, maybe this could mean that the app is not as

user friendly as it could be, as its use is discouraging customers from activating their account.

- Event 11
 - Finally, add_to_cart has a positive correlation with account activation. This is a promising feature as that means that customers who simply add something to their cart are more likely to activate their account.

Some further analysis shows that only 22.8% of the customers who added to their cart ended up activating their account, whereas 99.8% of those who placed a downpayment ended up activating their account. This is a huge dropoff and presents an opportunity for Fingerhut to step in once an item is added to the customers cart in order to incentivize the checkout process which is the path to account activation.

Finally, we created heat maps of the transition probability from one state to the next for accounts that were activated, and ones that were not activated to further inspect the relationships between features.



From the heatmaps, we can see that application_web_view is in fact a bottleneck. It rarely leads to another event other than itself in both graphs, meaning the customer gets stuck in

that step and doesn't progress. This validates our negative correlation from above. Additionally, the event with the highest probability of transitioning to event 29 (activation) is event 28 (order_shipped), event 27 (account_downpaymentcleared). These make sense with the definition that we know of activation preceding a purchase.

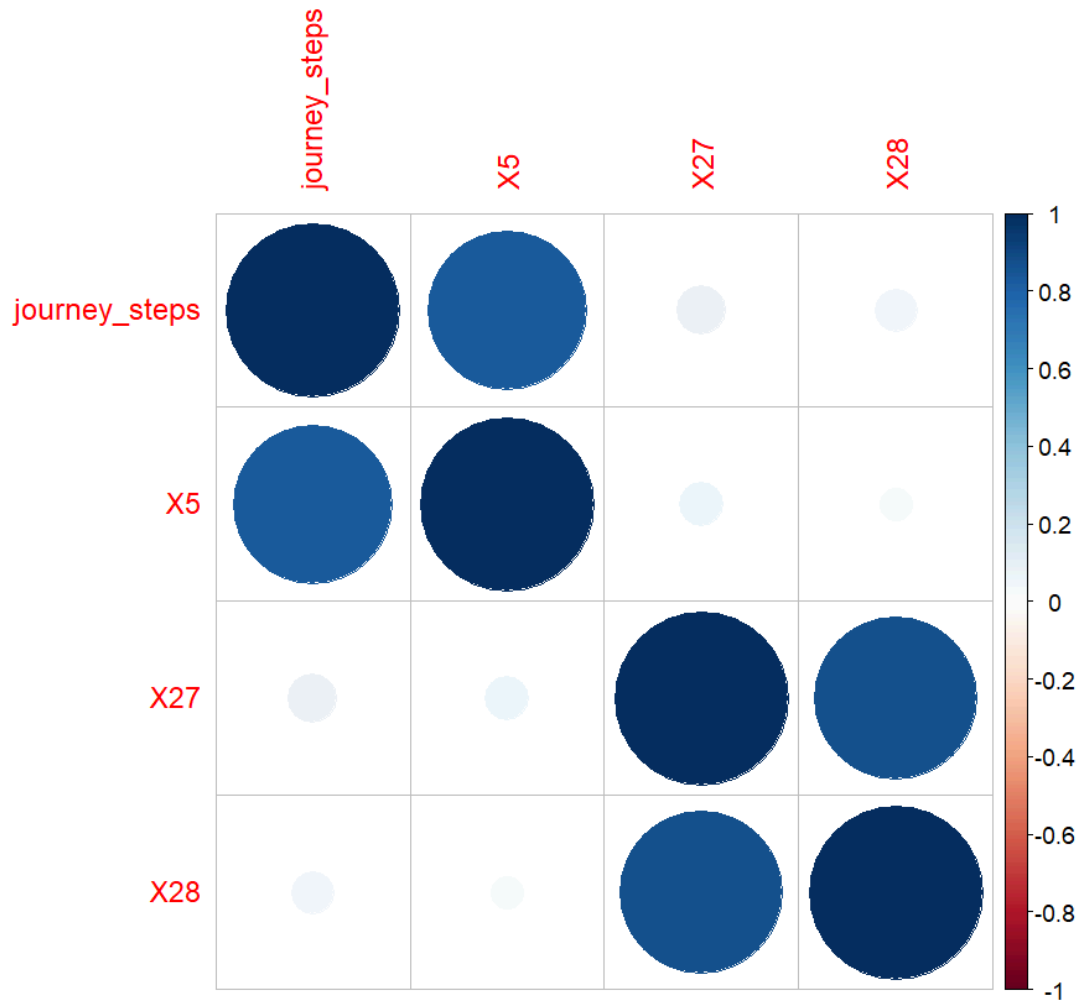
In conclusion, the events that have the highest positive correlation with activation are place_downpayment, place_order_web and add_to_cart. Each of these steps could have been expected. The most interesting insight from this analysis comes from the events which are negatively correlated. The two most negatively correlated events are catalog_mail, and application_web_view respectively. These are surprising as catalog_mail is meant to increase a customer's chance of purchase. Furthermore, the negative correlation and apparent bottleneck at application_web_view suggest a need for Fingerhut to take a deeper look at the user experience of their application and how to encourage customers to move past simply opening the app.

→ Optimized Customer Journey

Recall that our definition for an optimized customer journey would be the shortest journey that would connect a customer's activation milestone to their product purchase milestone. To accomplish this, we wanted to find which event would elongate the customer journey and which would shorten this. And so, a linear regression model was fitted to the amount of journey steps each customer had, with the predictors being the frequency of the event id's. The model's coefficients would then be extracted to interpret the weights each event has on the customer's journey steps. The following snippet shows a preview of the layout of the data inputted into the model.

	journey_steps	X1	X2	X3	X4	X6
1	22	1	1	1	8	1
2	35	1	0	1	9	5
3	15	1	0	1	3	2
4	36	3	0	1	13	1
5	14	2	0	0	1	1
6	38	0	1	1	18	1
7	29	1	1	1	6	1
8	6	1	0	0	2	1
9	16	0	1	0	8	1
10	20	2	0	0	2	4

Afterward, some of the preliminary analysis included looking at the distribution of events for each customer, variances, and more. Collinearity was a concern because of the results of the following plot:



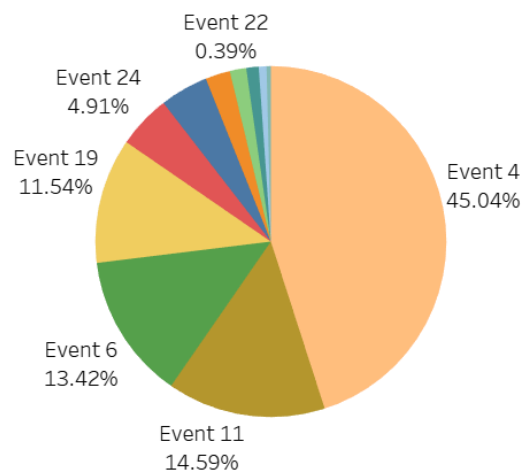
From this, we see event 5 is highly collinear with the response variable, and events 27 and 28 are collinear with one another too. In addition, their corresponding VIF's (Variance Inflation Factor) were high. For the model's performance, we will omit these variables. However, these correlations portray analysis that will be discussed in this section's conclusion.

For the data, we are working with customers that are activated and purchased (this implies events 7 or 18 exist and 12, 15, or 29 exist). In addition, the model will omit these variables because of redundancy. Also, events 10 and 20 do not exist for any customer. After normalizing the event frequencies, the model was fitted and predictions were extracted. The following are the metrics of the model:

- Mean Absolute Error - 2.61
- R Squared - 0.991
- Root Mean Squared Error - 4.55

These metrics imply that the model has good accuracy. The only caveat is that the RMSE is bad while MAE and R squared are good. This implies that the model is good but when it makes a mistake, usually its mistakes are largely off. Nevertheless, the general performance is good, so the features will be extracted. The following is a pie chart that summarizes the most impactful events to the customer journey length:

Event Types' Influence to Regression Algorithm



The following snippet shows the detailed breakdown of the model:

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>
1	(Intercept)	47.8	0.0741	645.	0
2	X4	33.1	0.112	296.	0
3	X6	11.1	0.0974	114.	0
4	X11	8.21	0.116	70.9	0
5	X19	6.33	0.0901	70.2	0
6	X24	2.89	0.0773	37.3	1.20e-265
7	X1	2.55	0.121	21.1	5.66e-94
8	X3	2.45	0.0891	27.5	2.14e-153
9	X8	1.00	0.0755	13.3	1.92e-39
10	X21	0.794	0.113	7.03	2.45e-12
11	X2	0.324	0.0772	4.19	2.80e-5
12	X22	0.178	0.0772	2.30	2.14e-2

In short, the coefficients show the behavior of journey steps when given an event. In other words, if the coefficient is greater than 1, the event tends to unnecessarily greatly increase the journey steps. If less than 1, it tends to slightly increase the journey steps. Note, intercept is insignificant for there cannot be 47.8 steps in a journey with no event frequencies.

→ Optimized Customer Journey: Conclusion

- Collinear Variables: 5 and journey_steps
 - This implies that a majority of customers' journey steps are dictated by *view_cart*.
 - Because of this, to shorten a majority of the customer journeys, event 5 must be limited.
- Important Events
 - The following events should be limited as possible to quicken the customer journey (in order of most impactful to least impactful):
 - 4, 6, 11, 19, 24, 1, 3, 8

- The following events do not have a strong impact on customer journey:
 - 22, 2, 21

XGBoost Algorithm

→ Further purchase and activation analysis

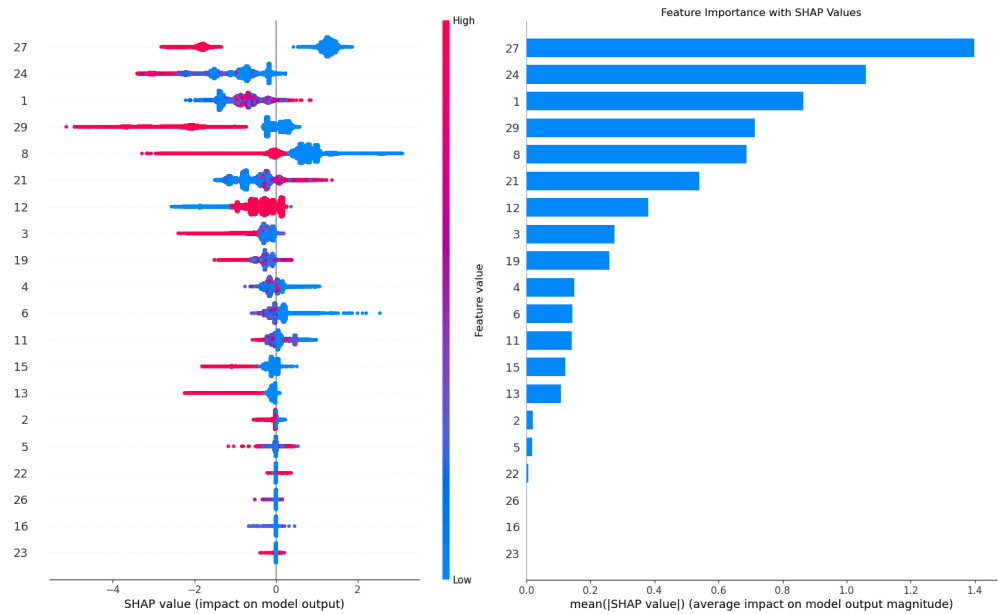
In this section, we implemented XGboost to more closely investigate the events that influence purchasing behavior and account activation. The analysis is segmented into two distinct components: the purchasing aspect and the account activation aspect. Additionally, in this section, we did not remove or omit any data point. We utilized all the data provided in the lecture to analyze our dataset with XGboost.

- **Purchase**

In the first part of our study, focusing on purchasing behavior, we divided the dataset into training and testing sets for the XGBoost model, randomly splitting 70% for training and 30% for testing based on the insights gathered from data overview and initial analysis. Before applying XGBoost for classification, it's crucial to eliminate certain events that directly indicate a purchase, to ensure the model does not use these as direct predictors of purchasing behavior. Specifically, event IDs No.7 and No.18, which denote product purchases via the website and by phone, respectively, are removed to prevent the model from relying on these clear indicators. Similarly, event No.28, related to order shipment, is also excluded because it inherently signifies that a purchase has occurred.

We configured the XGBoost model to predict a binary outcome: '0' for customers who made a purchase at Fingerhut and '1' for those who did not. Utilizing the

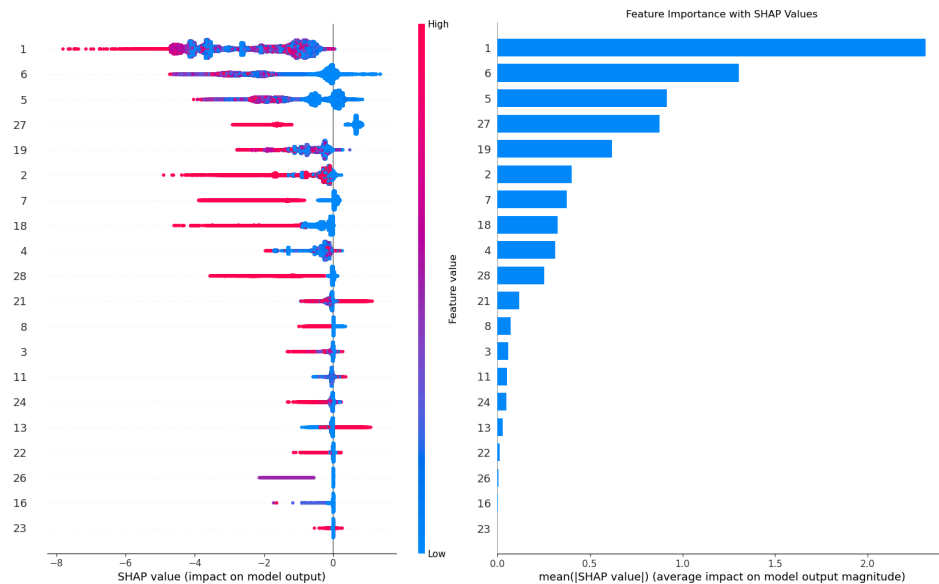
Python XGBoost framework, we achieved an impressive 97.1% accuracy on the test set. While XGboost excels as a supervised machine learning algorithm for classification, it does not automatically highlight the most influential features affecting predictions. To bridge this gap, we incorporated Shapley Additive exPlanations (SHAP) values, invented by Lloyd Shapley. SHAP values provide a robust interpretive framework that measures the contribution of each feature to individual predictions, greatly enhancing the model’s transparency. This approach fairly distributes the responsibility for the output across entire features, giving insightful observations about the model’s behavior at both local and global scales. The consistency of SHAP values ensures that features’ contributions are precisely captured, even if the model is more complicated. Also, SHAP values are not limited by the model and can be applied to wide machine learning approaches, including deep learning models.



The SHAP summary plot above illustrates the impact of different events on our binary classification model. We established that a SHAP value exceeding 0.5 is considered a significantly important feature within this dataset. The feature value here represents the frequency of each event for individual customers, essentially tracking how often each event occurs for them. For example, a red dot on the horizontal line for event 27 indicates customers with a high frequency of this event, which correlates with a negative SHAP value. This suggests that customers with more frequent interaction in event 27, which is related to clearing down payments, are more likely to make a further purchase (classified as 0). Conversely, a blue dot signifies customers with a low frequency of the event. When red dots are observed on the left side of the plot, it implies that a higher occurrence of the event is associated with an increased likelihood of purchasing (classified as 0). If red dots appear on the right side, it indicates the opposite: a higher occurrence corresponds with a decreased likelihood of purchase (classified as 1). Event 1, related to the distribution of promotion codes, suggests a significant effect on purchasing behavior; sending a moderate number of promotions appears to encourage purchases, while an overabundance might render them less effective. This could be due to customers perceiving that discounts are always readily available, thus reducing the immediate incentive to make a purchase. Consequently, customers who receive an excessive number of promotion codes may be less likely to complete a purchase. Event 24, involving customers clicking on campaign emails, clearly shows that those who engage more with campaign emails are more likely to purchase products than those who

click on few or no campaign emails. Similarly, event 29, account activation, indicates that customers who activate their accounts are more inclined to purchase, with no significant impact observed for non-purchases. Event 8, related to placing down payments, suggests that more frequent down payments are indicative of a willingness to purchase, although it's not a definitive predictor since some high-frequency down payments align with a SHAP value of 0, indicating no association with the outcome. Event 21, which tracks the receipt of catalog emails, indicates a potentially inverse relationship between the frequency of emails and the likelihood of making a purchase. A moderate volume of catalog emails may incentivize purchases, but an excessive amount appears to have the opposite effect, potentially leading to customer fatigue and a decreased propensity to buy. In essence, customers bombarded with too many catalog emails may be less inclined to make a purchase compared to those who receive them less frequently.

- **Activation**



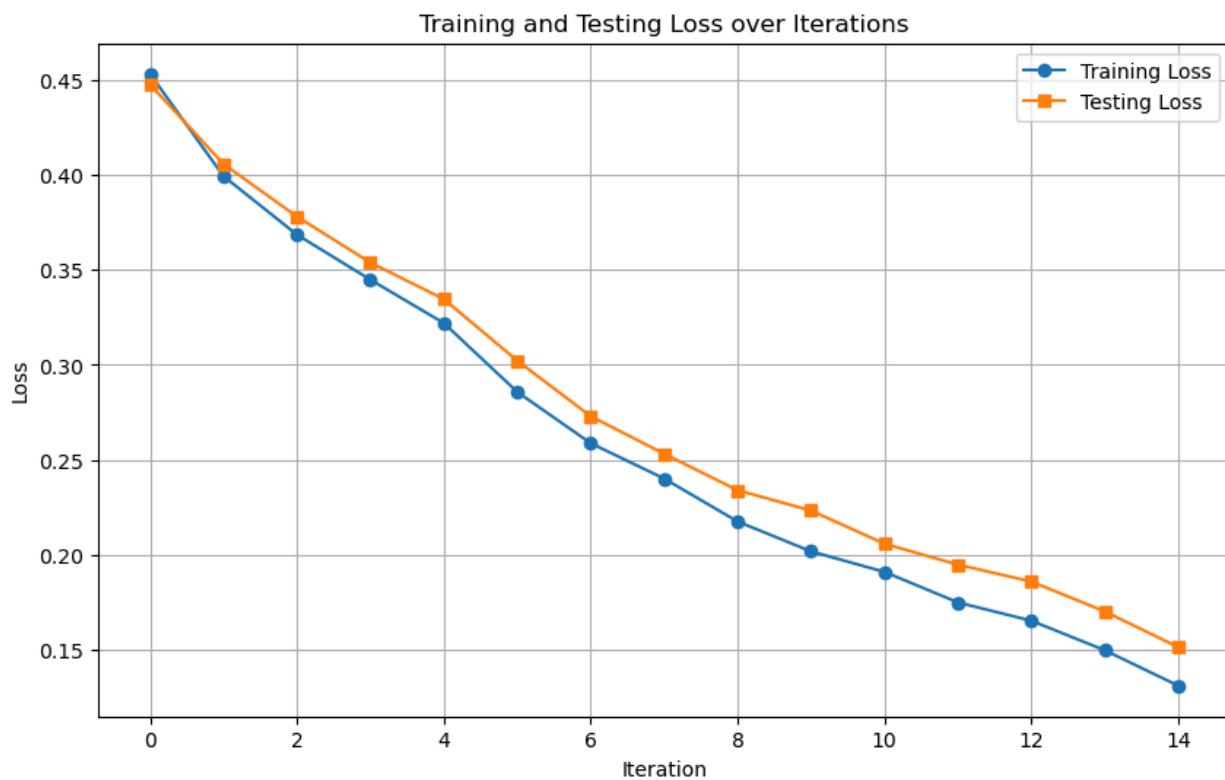
We turn our attention to account activation, a step that occurs after purchasing but before the placement and clearance of down payments. Events 12, 15, and 29 have a direct association with account activation. To delve deeper into how other, less directly related events influence account activation, we have removed these particular events from our dataset. This allows us to explore the effects of other customer interactions on the likelihood of activating an account. The accuracy rate of the XGboost classifier of account activation is approximately 97% on the test dataset, which is exceptionally high. In the second part, which concentrates on account activation, the SHAP summary plot above indicates that event No.1, 'promotion created', has the greatest impact on activation. Individuals with a high frequency of promotions are more inclined to activate their accounts, suggesting even a moderate level of promotion activity can positively influence activation. Event No.6, 'begin_checkout', is another significant predictor of account activation, where a higher occurrence implies a greater likelihood of the customer proceeding to activate their account. Similarly, event No.5, 'view_cart', demonstrates that increased engagement with the cart correlates with a higher propensity for account activation. Event No.27, 'down payment cleared', is also a key determinant of activation. Intuitively, customers clearing down payments are likely to be engaged users, which often translates to account activation. The prominence of event No.1 is logical, as promotions may be exclusive to activated accounts, incentivizing customers to engage further. Events No.5 and No.6 may at first appear unrelated, but purchasing behavior, initiated by actions such as checking out and viewing the cart, is often a precursor to account activation, especially for new users. Event No.27 is straightforward; clearing down payments typically necessitates an active account.

RNN (Recurrent Neural Network)

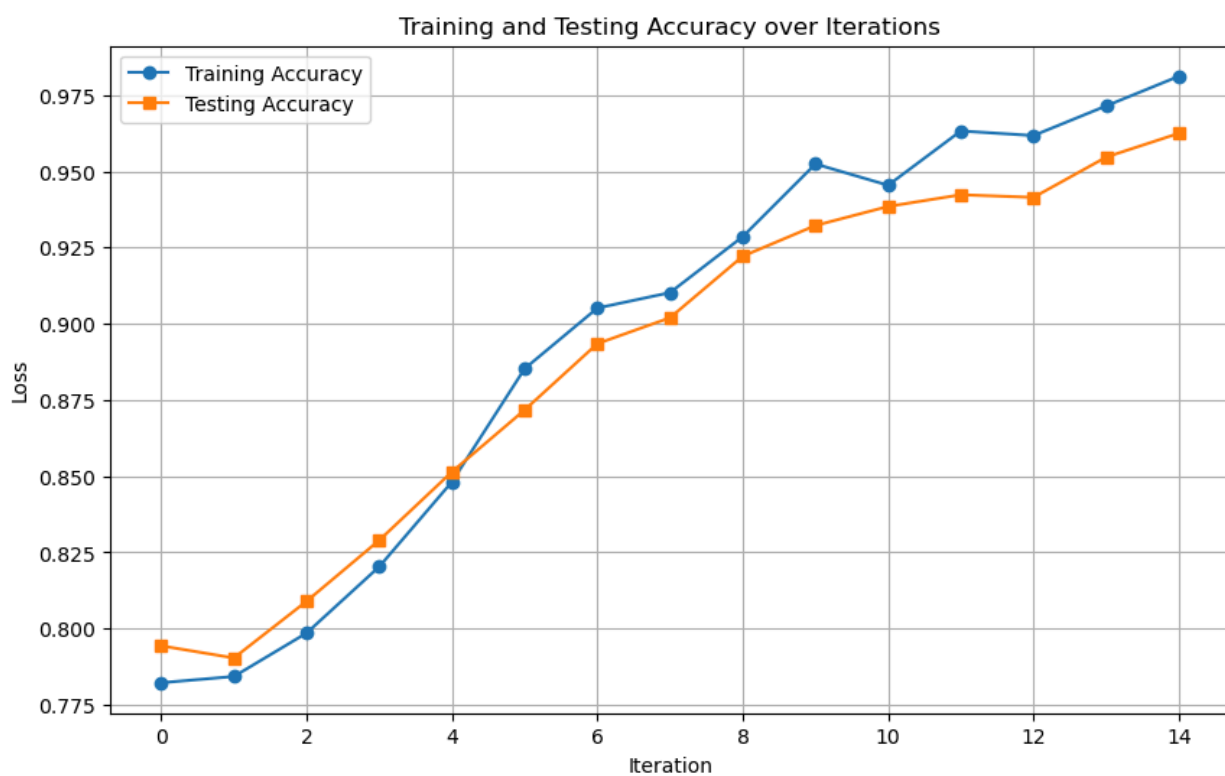
In this section, we introduced a RNN model to evaluate the performance of other models. But firstly we would like to introduce the notion of what an RNN model is: RNN is an extension of the traditional neural network with a recurrent loop inside the hidden layer, designed to effectively capture temporal dynamic behaviors. This makes it particularly suitable for analyzing time series data, such as stock prices. Since the journeys of each account follows a time series data, it is possible to implement an RNN within this framework.

Since the original dataset is of great length, and an RNN would be time-consuming with an extremely large dataset due to CPU/GPU constraint, subsampling is implemented in our model. In fact, when the sample size gets to the scale of 10k, the RNN model stops responding. However, reducing subsample size introduced extreme variance into the RNN model. Even after cleaning the data, the set has 55,853,910 observations, so a single subsampling of 5,000 observations is sufficient. To address this issue, our RNN model introduced the bagging method. First, several RNN models are trained based on subsampling of the original dataset to make several weak models. After the training step is completed, all the models are assembled together to make the final predictions based on the output of all the models. The weak models learn different features from the original dataset, and once aggregated, contribute to the diminishing of the overall variance across the models. (For preprocessing code: refer to the preprocessing.ipynb. For construction of the RNN model: refer to the RNN_bagging.ipynb)

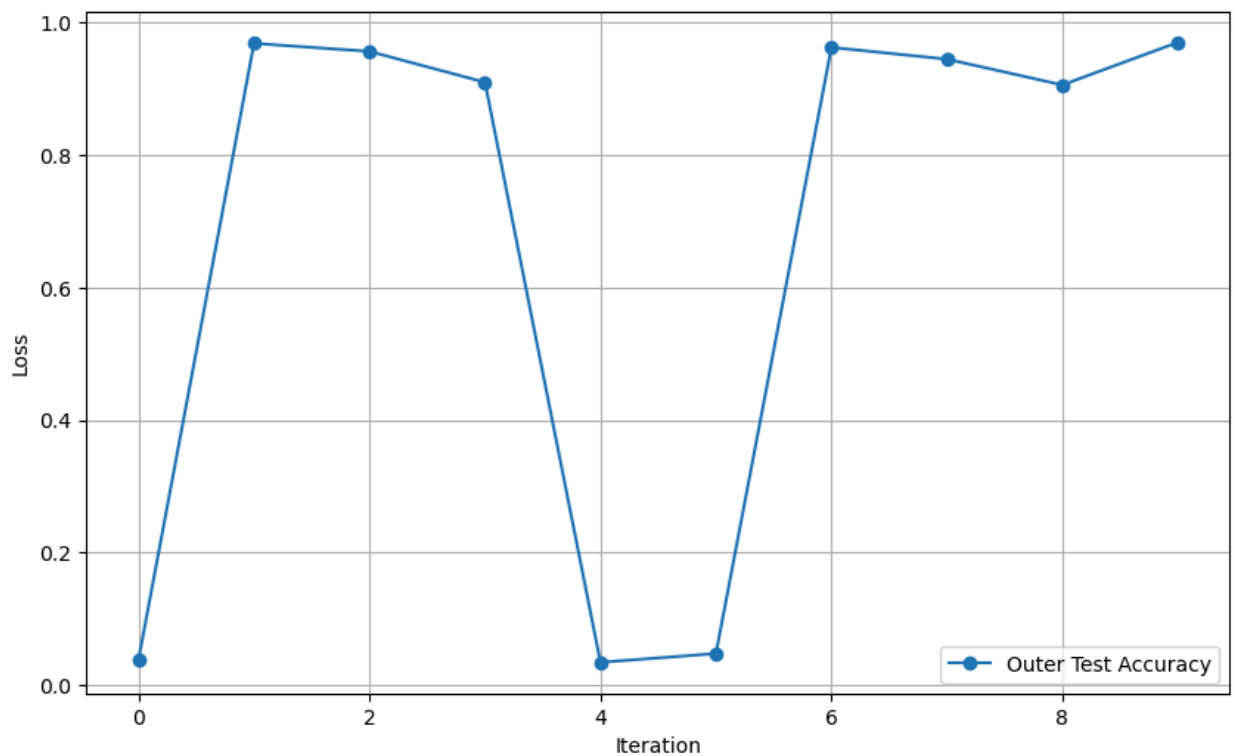
To evenly reduce the overall variance, our group designed a training and testing loop, using mini-batch with batch size of 128 sampling to reduce possible variance within our sampled data. After performing 15 epochs, we get the training loss and test loss as follows:



With the corresponding training and testing accuracy:



Reducing training loss and improving accuracy indicate that the RNN model is accurately learning from the data and generalizing well. The accuracy is high – around 97.5%. This high accuracy was achieved in part due to the bagging method implemented in the model. However, as indicated by the plot below, there is still an issue with high variance:



This plot evaluated the performance of the entire RNN model based on the 10th iteration of external evaluation each with a sample size of 3000. Most of the iterations have high accuracy, but 3 of them have accuracy near 0. This could be due to overfitting of the RNN model, even when the bagging method is

This is a plot for evaluation of the whole RNN model based on 10th outer evaluation, each with a sample of 3000. Noticing that most of the evaluations have a high accuracy, but 3 of them seem to have accuracy almost to 0. This may be due to the overfitting of the RNN model, even when the bagging method is considered.

Conclusion

- **LUKE'S ANALYSIS RESULTS**

- Fingerhut should increase the frequencies of the following events for each customer to have a better chance for customer purchases (ordered from most to least impactful):

- 8, 29, 27, 24

- Fingerhut should decrease the frequencies of the following events for each customer to have a better chance for customer purchases (ordered from most to least impactful):

- 21, 1, 22

- The following events should be limited as possible to quicken the customer journey (in order of most impactful to least impactful):

- 4, 6, 11, 19, 24, 1, 3, 8

- The following events do not have a strong impact on customer journey:

- 22, 2, 21

- Collinear Variables: 5 and journey_steps

- This implies that a majority of customers' journey steps are dictated by *view_cart*.

- Because of this, to shorten a majority of the customer journeys, event 5 **MUST** be limited.

Further Discussion

- Based on the conclusions of the models, certain events' frequencies should be increased and some decreased. Of course, some of these events can easily be edited by a company decision. However, for decisions that have a longer lasting effect, A/B testing should be implemented to see if those changes should be implemented to the entire customer base of Fingerhut.
- For the constructed RNN binary prediction model, due to the constraint of computational resources, sub-sampling has to be implemented. Though the bagging method could help to reduce the variance, considering the huge amount of original data, bagging may not perform so well as desired. Possible further steps would be to stratify the whole data set first, and then do bagging based on the stratified original data, and then ensemble all the results to reduce variance. LSTM can also be implemented rather than RNN, since it would be efficient in remembering the long term sequential data. A possible solution file has been uploaded as a supplementary file, with less variance in the overall data.

References

For XGboost and extracting feature importance (SHAP):

- <https://towardsdatascience.com/interpretable-machine-learning-with-xgboost-9ec80d148d27>
- <https://medium.com/@emilykmarsh/xgboost-feature-importance-233ee27c33a4>
- <https://towardsdatascience.com/explainable-ai-xai-with-shap-multi-class-classification-problem-64dd30f97cea>
- <https://www.kaggle.com/code/tunguz/tps-02-21-feature-importance-with-xgboost-and-shap>
- <https://medium.com/sfu-csmp/xgboost-a-deep-dive-into-boosting-f06c9c41349>
- <https://towardsdatascience.com/a-beginners-guide-to-xgboost-87f5d4c30ed7>
- MATH 118 notes/source codes

For Apriori algorithm:

- Apriori Implementation:
 - <https://towardsdatascience.com/apriori-association-rule-mining-explanation-and-python-implementation-290b42afdfc6>

For Multivariate classification:

- Application to the Customer Journey:
 - https://www.scs-europe.net/dlib/2021/ecms2021acceptedpapers/0048_mlbdcms2021_0030.pdf

For building an RNN model:

- RNN basic:
 - <https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>
- Bagging implementation in RNN:
 - <https://www.datacamp.com/tutorial/what-bagging-in-machine-learning-a-guide-with-examples>
 - <https://www.simplilearn.com/tutorials/machine-learning-tutorial/bagging-in-machine-learning>